

# Analysis of supply chain

Check the download source: verify the vendor, total number of downloads, download and upload dates, the actual model size, and the downloaded model size.

Then verify the model using Python tools. Do **not** use `pickle.load()` ; instead, use `pickletools` to inspect the model first.

```
python3 -m pickletools model.pkl 2>&1 | head -30
```

## Red flags checklist for any pickle file:

| Pattern                                  | Concern Level | Legitimate Use?                                           |
|------------------------------------------|---------------|-----------------------------------------------------------|
| <code>os</code>                          | Critical      | Almost never in a model file                              |
| <code>system</code> , <code>popen</code> | Critical      | Never                                                     |
| <code>subprocess</code>                  | Critical      | Never                                                     |
| <code>socket</code>                      | Critical      | Never                                                     |
| <code>eval</code> , <code>exec</code>    | Critical      | Never                                                     |
| <code>curl</code> , <code>wget</code>    | Critical      | Never                                                     |
| <code>STACK_GLOBAL</code>                | Moderate      | Common in legitimate pickles, too; check what it resolves |
| <code>REDUCE</code>                      | Moderate      | Common in legitimate pickles; check context               |

can use this python code for the analysis

```
#!/usr/bin/env python3
"""
Safe Pickle Analysis Tool
Structured summary of pickle file safety.
Uses pickletools to inspect without loading.
Output format matches Room 2 expected output exactly.
Usage: python safe_analysis.py <pickle_file>
"""
```

```
import pickletools
import sys
```

```
import os
from pathlib import Path
```

```
DANGEROUS_MODULES = {"os", "subprocess", "sys", "socket",
"shutil", "ctypes",
"builtins", "importlib", "code", "codeop"}
DANGEROUS_FUNCTIONS = {"system", "popen", "exec", "eval",
"execfile",
"call", "Popen", "check_output", "run",
"getattr", "setattr", "delattr", "import"}
SUSPICIOUS_PATTERNS = ["http://", "https://", "ftp://", "cu
r l ", "wget ",
"/etc/passwd", "/etc/shadow", "/tmp/", "$(", "`"]
```

```
def analyze_pickle(filepath):
    """Produce the structured safety summary expected by Room
    2."""
    path = Path(filepath)
    size_bytes = path.stat().st_size
```

```
# Human-readable size (use decimal MB to match Room 2 conve
ntions)
if size_bytes >= 1_000_000:
    size_str = f"{size_bytes / 1_000_000:.1f} MB"
elif size_bytes >= 1000:
    size_str = f"{size_bytes / 1000:.1f} KB"
else:
    size_str = f"{size_bytes} bytes"

with open(filepath, "rb") as f:
    data = f.read()

ops = list(pickletools.genops(data))

# — Collect findings —
dangerous_opcodes = []
```

```

suspicious_strings = []
imported_modules = []
pending_global = None # tracks module.function for STACK_G
LOBAL

i = 0
while i < len(ops):
    opcode, arg, pos = ops[i]
    name = opcode.name

    # Detect STACK_GLOBAL pattern: two SHORT_BINUNICODE the
n STACK_GLOBAL
    if name in ("SHORT_BINUNICODE", "BINUNICODE") and isins
tance(arg, str):
        # Check if this is a module import
        if arg in DANGEROUS_MODULES:
            imported_modules.append(arg)
            # Look ahead for the function name + STACK_GLOB
AL

            if i + 2 < len(ops):
                next_op, next_arg, _ = ops[i + 1]
                # Skip MEMOIZE between module and function
                func_idx = i + 1
                while func_idx < len(ops) and ops[func_idx]
[0].name == "MEMOIZE":
                    func_idx += 1
                if func_idx < len(ops):
                    func_op, func_arg, _ = ops[func_idx]
                    if func_op.name in ("SHORT_BINUNICODE",
"BINUNICODE"):
                        # Look for STACK_GLOBAL after the f
unction name

                        sg_idx = func_idx + 1
                        while sg_idx < len(ops) and ops[sg_
idx][0].name == "MEMOIZE":
                            sg_idx += 1
                            if sg_idx < len(ops) and ops[sg_id
x][0].name == "STACK_GLOBAL":

```

```

                                pending_global = f"{arg}.{func_
arg}"

                                dangerous_opcodes.append(
                                    f"STACK_GLOBAL: {arg}.{func
_arg}"

                                )

                                # Check for suspicious string content
                                for pattern in SUSPICIOUS_PATTERNS:
                                    if pattern in arg:
                                        suspicious_strings.append(arg)
                                        break

                                if name == "REDUCE" and pending_global:
                                    dangerous_opcodes.append(
                                        f"REDUCE: executes {pending_global} with argume
nts"
                                    )
                                    pending_global = None

                                i += 1

# — Output —
print(f"=== Pickle Safety Analysis ===")
print(f"File: {filepath}")
print(f"Size: {size_str}")

if dangerous_opcodes:
    print(f"\nDangerous opcodes found:")
    for entry in dangerous_opcodes:
        print(f"  [CRITICAL] {entry}")

if suspicious_strings:
    print(f"\nSuspicious strings:")
    for s in suspicious_strings:
        print(f"  [CRITICAL] '{s}'")

# Verdict

```

```

if dangerous_opcodes or suspicious_strings:
    # Build a concise description of what was found
    targets = [m for m in imported_modules if m in DANGEROUS_MODULES]
    target_str = ".".join([targets[0], "system"]) if targets else "unknown"
    print(f"\nVerdict: UNSAFE - Contains executable code targeting {target_str}")
else:
    print(f"\nNo dangerous opcodes or suspicious strings found.")
    print(f"\nVerdict: SAFE - No executable code detected")

```

```

def main():
    if len(sys.argv) < 2:
        print("Usage: python safe_analysis.py <pickle_file>")

        print("\nExamples:")
        print("  python safe_analysis.py /opt/supply-chain/models/code_reviewer.pkl")
        print("  python safe_analysis.py /opt/supply-chain/models/code_reviewer_v1.pkl")
        sys.exit(1)

```

```

filepath = sys.argv[1]
if not Path(filepath).exists():
    print(f"\n[!] Error: File not found: {filepath}\n")
    sys.exit(1)

analyze_pickle(filepath)
if name == "main":
    main()

```

## Examine Suspicious Dependencies

A malicious package can be present inside the requirement.txt or Typosquatted Version can also be same harmful because it may contain a malicious payload to damage or steal information

Examine prompt template don't set it to auto-update and verify before deploying

| Vector                         | Mechanism                                               | Detection Difficulty                     | Impact                                             |
|--------------------------------|---------------------------------------------------------|------------------------------------------|----------------------------------------------------|
| Pickle <code>__reduce__</code> | Embeds code in model files                              | Moderate: pickletools can reveal         | Code execution on model load                       |
| Keras Lambda/custom layers     | Embeds code in model architecture                       | Moderate: architecture inspection        | Code execution at inference time                   |
| Dependency confusion           | Hijacks internal package names                          | Low: version anomalies visible           | Code execution on pip install                      |
| Typosquatting                  | Misspelt package names                                  | Low: name comparison reveals             | Code execution on pip install                      |
| Repository manipulation        | Fake orgs, professional model cards                     | Moderate: reputation signals             | Trusted distribution of malicious models           |
| API provider compromise        | Silent updates, key exposure, prompt template tampering | High: no file artefact to scan           | Invisible model substitution or data exfiltration  |
| GGUF weight-level              | Backdoor fine-tuned into weights before quantisation    | High: no static analysis tools available | Triggered misclassification or output manipulation |